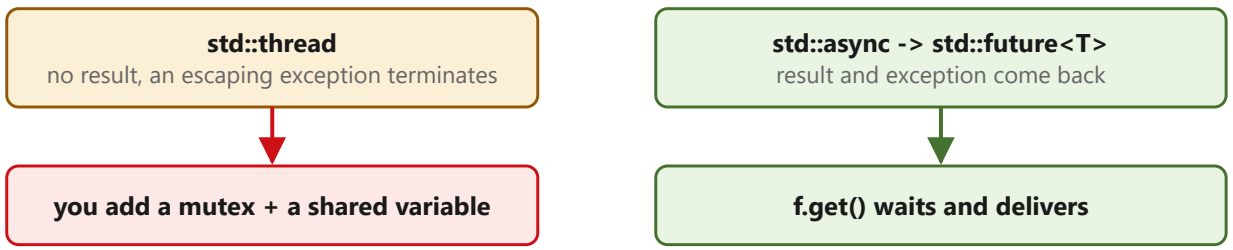
Futures, async and Promises

Running work in the background and getting the result back: `std::async` and `std::future`, exceptions that travel, `promise`, `shared_future` and `packaged_task`. Assumes Condition Variables.

Why tasks instead of raw threads

A `std::thread` runs a function but gives nothing back. To return a value you would need a shared variable, a mutex and a signal; to report an error you would need to catch it in the thread. A **task** packages all three: it runs a function, **delivers the result later** through a `std::future<T>` and **passes any exception** on to whoever asks for the result.



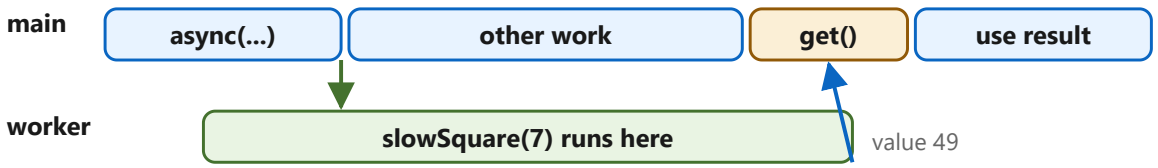
Tasks bundle the work, the result channel and the error channel.

	`STD::THREAD`	`STD::ASYNC` / TASK
result	none: use a shared variable	`future.get()`
exception in the function	`std::terminate`	rethrown by `get()`
when the work is not yet done	`join()` only	`wait()`, `wait_for()`, `get()`
control	full: pinning, detach	less: the library chooses

PREFER TASKS Use tasks for "compute this and give me the answer". Use a raw thread for a long-lived worker you manage yourself.

`std::async` and `std::future`

`std::async` starts the function, normally on another thread, and returns a `std::future`. Call `get()` when you need the answer.



If the task is faster than the other work, get() returns at once; if slower, main waits.
get() can be called once; wait() and wait_for() do not take the value.

The future lets main overlap its own work with the task.

```
// starts now
std::future<int> f = std::async(std::launch::async, slowSquare, 7);
// main keeps going
doOtherWork();
// waits if needed: 49
int result = f.get();
```

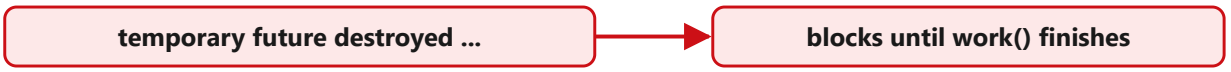
CALL	DOES
`f.get()`	waits, then returns the value (or rethrows), once
`f.wait()`	waits, does not take the value
`f.wait_for(100ms)`	waits up to a time: `ready`, `timeout` or `deferred`
`f.valid()`	true until `get()` has been called

SAY LAUNCH::ASYNC The default policy may *defer* the call until `get()`, so nothing runs in parallel. Pass `std::launch::async` when you want a thread.

The future that waits in its destructor

One surprise catches almost everyone. The future returned by `std::async` **blocks in its destructor** until the task is finished. If you throw the future away, the call is silently synchronous.

```
async(launch::async, work); // result discarded
```




```
auto f = async(launch::async, work); // kept
```



Always keep the future, or the task runs to completion before the next line.

FUTURE FROM	DESTRUCTOR
`std::async` (launch::async)	**blocks** until the task completes
`std::promise`, `packaged_task`	does not block

This is why `std::async` is not a drop-in fire-and-forget. For work that must run without being waited on, use a pool or a detached design, and think about shutdown.

Exceptions travel with the result

An exception in a plain thread kills the program. In a task it is **caught, stored and rethrown** when you call `get()`, so error handling stays where the result is used.



Without a future, the same throw would call std::terminate.

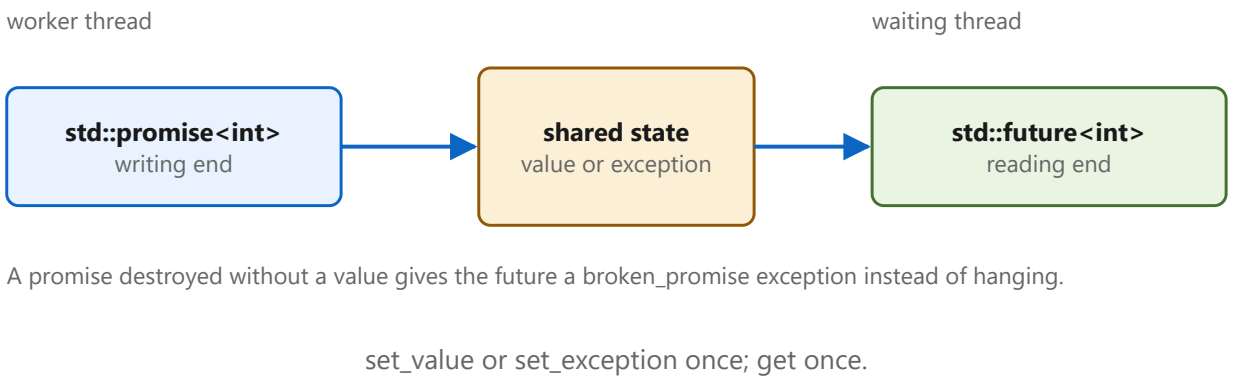
Catch it where you call get(), exactly as if the function ran in your thread.

Errors cross the thread boundary through the future.

```cpp
auto f = std::async(std::launch::async, [] { throw std::runtime_error("bad");
return 1; });
// the exception comes out of get()
try { f.get(); }
// prints: bad
catch (const std::exception& e) { std::cout << e.what(); }
```

`promise` : the writing end

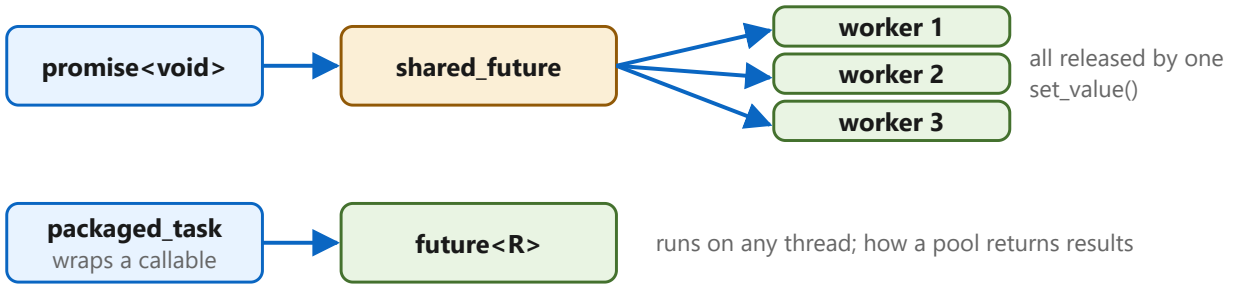
Sometimes you manage the thread yourself and still want a result channel. A `std::promise` is the writing end: the worker calls `set_value` (or `set_exception`) once, and the waiting thread gets it from the matching `future`.




```cpp
std::promise<int> p;
std::future<int> f = p.get_future();
// the worker fulfils the promise
std::thread t([&p] { p.set_value(42); });
std::cout << f.get();                          // 42
t.join();
```

`shared_future` and `packaged_task`

Two more tools complete the family.



Broadcast with shared_future, deliver results with packaged_task.

TYPE	ROLE
`std::promise<T>`	the **writing** end: `set_value` or `set_exception`, once
`std::future<T>`	the **reading** end: `get()` once; move-only
`std::shared_future<T>`	a **copyable** read end: several threads can wait on the same result
`std::packaged_task<R(Args)>`	wraps a callable and routes its result or exception into a future

```
std::promise<void> go;
std::shared_future<void> start = go.get_future().share();      // copyable
for (int i = 0; i < 4; ++i) workers.emplace_back([start] { start.wait();
run(); });
// all four are released together
go.set_value();
```

- Compiled: four workers waiting on a shared `promise<void>` had started 0 before `set_value()` and 4 after.
- A promise is **one-shot**. For a repeated signal use a condition variable; for many messages use a queue.

Waiting with a time limit

Waiting forever is rarely what you want. `wait_for` and `wait_until` return a status instead of blocking without limit.

RESULT OF `WAIT_FOR`	MEANING
`future_status::ready`	the value or exception is there: `get()` will not block
`future_status::timeout`	not ready yet: do something else and ask again
`future_status::deferred`	the task has not started: it runs on `get()` (default launch policy)

```
if (f.wait_for(std::chrono::milliseconds(100)) == std::future_status::ready)
    use(f.get());
else
    showSpinner();                    // still working: stay responsive
```

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Why tasks instead of raw threads	C++ Concurrency in Action	2.4 Choosing the number of threads at runtime	54 to 56
	C++ Concurrency in Action	4.2 Waiting for one-off events with futures (std::async, packaged_task, promise, shared_future)	104 to 115
	C++ Concurrency in Action	4.3 Waiting with a time limit	116 to 121
	C++ Concurrency in Action	4.4.7 to 4.4.10 Latches and barriers (the Concurrency TS versions of C++20 std::latch and std::barrier)	141 to 147
	C++ Concurrency in Action	8.1 Techniques for dividing work between threads	275 to 282
	C++ Concurrency in Action	9.1 Thread pools (a simple pool, waiting for tasks, work stealing in 9.1.5)	324 to 337
	C++ Concurrency in Action	9.2 Interrupting threads (the idea behind std::stop_token)	338 to 349
	C++ Concurrency in Action	10.1 to 10.3 Parallel algorithms: execution policies and the parallel standard algorithms	350 to 362
std::async and std::future	Effective Modern C++	Item 35, Prefer task-based programming to thread-based	259 to 263
	Effective Modern C++	Item 36, Specify std::launch::async if asynchronicity is essential	263 to 268
	Effective Modern C++	Item 38, Be aware of varying thread handle destructor behavior	276 to 280
	C++ Move Semantics	Chapter 15 Move semantics in standard library types	229 to 250
shared_future and packaged_task	C++ Concurrency in Action	4.2 Waiting for one-off events with futures (std::async, packaged_task, promise, shared_future)	104 to 115